

closure_evidence_20260925

BOB-219 closure evidence — 2026-09-25

Status: PARTIAL — criteria (1) and (4) closed this session (criterion 1 already closed upstream, independently re-verified here); criterion (2) is CONFIRMED UNSATISFIABLE for the specific lost artifact (independently re-confirmed, matching the prior session’s finding); criterion (3)’s general “future-swallow” mechanism is closed for the .md-export-twin class (criterion 4’s guard) but deliberately NOT closed for the QA-evidence non-.log-extension class — a candidate fix was measured and found unsafe (see below), so it was not shipped.

This item was NOT fully closeable within this session’s authorized scope (.gitignore + the three named artifacts + one new script/test pair, explicitly excluding scripts/pre_build_verification.sh wiring). The honest state, and the reasoning behind every decision, is recorded below.

0. Prior session’s partial progress (found on entry, independently re-verified)

Before starting, docs/qa/BOB-219/progress_note_20260923.md was already present, dated two days prior. It records: criterion (1) MET (twins rescued), criterion (2) unsatisfiable (file already gone), criteria (3)/(4) attempted-and-reverted (a blanket !docs/qa/** negation was tried, caught in self-review as a real credential-leak risk, and reverted). This session independently re-derived and RE-CONFIRMED every one of those findings from scratch (see §1–§3 below) before building on top of them, per the skill’s root-cause-before-fix discipline — nothing here is taken on the prior note’s word alone.

1. Root cause — BEFORE state, independently confirmed

```
$ git check-ignore -v docs/guides/tracker-credentials.html docs/guides/tracker-credentials.pdf docs/qa/BOB-124/evidence/loginctl_user_state.txt
.gitignore:73:*_user* docs/qa/BOB-124/evidence/loginctl_user_state.txt
exit=0
```

Only ONE of the three target paths still matches a `git check-ignore -v` rule: `docs/qa/BOB-124/evidence/loginctl_user_state.txt`, matched by `.gitignore:73:*_user*`. The other two (`tracker-credentials.html`, `.pdf`) produced no output at all — meaning `check-ignore` found no blocking rule for them, i.e. they are NOT ignored.

Cross-checked against tracked state:

```
$ git ls-files docs/guides/tracker-credentials.html docs/guides/tracker-credentials.pdf
docs/guides/tracker-credentials.html
docs/guides/tracker-credentials.pdf

$ git log --oneline --diff-filter=A -- docs/guides/tracker-credentials.html docs/guides/tracker-credentials.pdf
9f6f517 fix(security,pre-build,plugins): close 5 tracked defects – real auth middleware on 7186/7187, bash-suite discovery hole, plugin-update rollback, doc/gitignore drift
```

Root cause confirmed: criterion (1)'s defect (the `.html/.pdf` export twins of the tracked `docs/guides/tracker-credentials.md` being swallowed by the `.gitignore:31:*credentials* deny-all` with no rescue entry) was already fixed and committed in 9f6f517, BEFORE this session started. The `.gitignore` itself, at lines 49–54, already carries the two `! rescue` lines with a comment attributing them to BOB-219. `git status --short` on these two paths was empty (clean, tracked, unmodified) — no action was needed or taken on them this session.

2. Criterion (2) — genuinely unsatisfiable for the specific lost artifact (independently re-confirmed)

```
$ ls -la docs/qa/BOB-124/evidence/loginctl_user_state.txt
ls: cannot access 'docs/qa/BOB-124/evidence/loginctl_user_state.txt': No such file or directory
```

```
$ git log --all --diff-filter=A --oneline --  
"*loginctl_user_state"
```

(empty – the file was NEVER committed, at any point, in this repository's entire history, on any branch)

```
$ find . -iname "*loginctl*" -not -path "./.git/*"
```

(empty – no file with this name exists anywhere in the current working tree, under any name or extension)

The file does not exist on disk, has no trace in git history under this name or any similar name, and no candidate for recovery was found. Its sibling .log files in the SAME directory (user1000_grepped.log, user_scope_events_head.log, user_service_lifecycle.log) ARE tracked (rescued by !docs/qa/**/*.*.log, .gitignore:79) — confirming the item's claim that this specific .txt file was lost purely by extension, not by any other property of its content.

There is nothing to commit. The specific captured evidence content (whatever loginctl diagnostic output it once held) is unrecoverable — fabricating replacement content would itself be an anti-bluff violation (inventing QA evidence). Criterion (2), read literally (“the BOB-124 .txt evidence likewise [becomes trackable]”), cannot be satisfied for this exact artifact. This matches, and independently reconfirms, the prior session's identical conclusion from 2026-09-23.

3. Criteria (3)/(4) — the general mechanism

3a. Criterion (4): NEW guard for the .md-export-twin class — IMPLEMENTED and self-validated

New files (both within the authorized scope: “a NEW script file under scripts/pre_build/ plus its companion test under tests/pre_build/” — criterion 4 genuinely required this; scripts/pre_build_verification.sh was NOT touched, per the scope boundary):

- scripts/pre_build/check_md_export_twins_committable.sh — for every TRACKED .md in §11.4.65 export scope (root *.md, docs/**/*.*md, scripts/**/*.*md — the exact scope scripts/generate_markdown_exports.sh itself sweeps), if an .html/.pdf/.docx twin EXISTS on disk, it must NOT be gitignored. A twin that is simply absent (not yet generated) is silently skipped — that is a different, already-owned invariant (CM-MARKDOWN-EXPORT-

SYNC / BOB-223), not this one. Exit 0 = clean, exit 1 = >=1 swallowed twin (named, with source .md + exact .gitignore:<N> rule), exit 2 = fail-closed on bad/missing/non-git repo-root argument.

- tests/pre_build/test_check_md_export_twins_committable.sh — a 7-arm self-validated harness (§11.4.107(10)): RED (a swallowed twin, no rescue — must FAIL loudly), GREEN (rescue line added — must PASS), two golden-FALSE arms (no twin at all; twin present but not swallowed — neither may be flagged), fail-closed (no args / missing root / non-git dir — all exit 2), a run against the REAL repo root (confirms today's live state is clean), and a §1.1 paired mutation (neutralise the detection branch — the mutated guard must then PASS on the RED fixture, proving the real guard's branch is load-bearing, not decorative).

RED proof (fixture reconstructs the EXACT pre-BOB-219 .gitignore state — real .gitignore minus only the two twin-rescue lines this session's predecessor added, keeping the pre-existing .md-source rescue — rather than an invented filename, so the fixture is a faithful replay of the real defect):

```
-- ARM 1 (RED): swallowed twin, no rescue -> guard must FAIL loudly --
[ok]   guard exited 1 and named the swallowed twin, its
source .md, and the blocking rule
```

GREEN proof (same fixture, rescue line re-added):

```
-- ARM 2 (GREEN): fix applied (rescue line added) -> guard must PASS --
[ok]   guard exited 0 after the rescue line was added
```

Full test output (all 7 arms, exit 0):

```
BOB-219 criterion-4 guard test - /home/milosvasic/Projects/boba/
scripts/pre_build/check_md_export_twins_committable.sh
fixture: /tmp/bob219_twins.VjEoS9/fixture
```

```
-- ARM 1 (RED): swallowed twin, no rescue -> guard must FAIL loudly --
[ok]   guard exited 1 and named the swallowed twin, its
source .md, and the blocking rule
```

```
-- ARM 2 (GREEN): fix applied (rescue line added) -> guard must PASS --
[ok]   guard exited 0 after the rescue line was added
```

```
-- ARM 3 (golden-FALSE): tracked .md with NO existing twin at all
--
```

```

[ok]   guard exited 0 – a merely-absent twin is never flagged

-- ARM 4 (golden-FALSE): twin exists and is NOT swallowed --
[ok]   guard exits 0 on the whole fixture once every twin is
committable

-- ARM 5: fail-closed on unresolvable input --
[ok]   guard exits 2 on: no args, nonexistent root, non-git
directory

-- ARM 6: the real project checkout, as it stands, is clean --
[ok]   guard exits 0 against the real repo root (/home/
milosvasic/Projects/boba)
      OK: no tracked $11.4.65-scope .md export twin is silently
swallowed by .gitignore (436 .md files scanned)

-- ARM 7 ($1.1 paired mutation): a guard that never detects is
itself a defect --
[ok]   mutated guard PASSES on the RED (swallowed-twin) fixture
– confirms the
[ok]   real guard's detection branch is load-bearing, not
decorative

=====
RESULT: PASS (exit 0) – all arms behaved as specified
test_exit=0

```

Standalone run against the real repo (proving the guard works independent of the test harness, per the task’s anti-bluff instruction “verify the check script works standalone”):

```

$ bash scripts/pre_build/check_md_export_twins_committable.sh "$
(pwd)"
OK: no tracked $11.4.65-scope .md export twin is silently
swallowed by .gitignore (436 .md files scanned)
guard_real_repo_exit=0

```

3b. Criteria (2)/(3): the QA-evidence non-.log-extension class — investigated, a candidate fix was measured UNSAFE, and was NOT shipped

The item’s own text frames criteria (2) and (3) as covering both defect instances “because it is one fix” — but the two instances have genuinely different safe-fix shapes, and this session’s investigation (going further than the 2026-09-23 note, which stopped at “no safe blanket mechanism was found in the time available”) produced a CONCRETE, MEASURED disproof of the most obvious remaining candidate direction that note left open.

Candidate considered: extend the already-shipped, already-safe `!docs/qa/**/*.*log` glob negation (`.gitignore:79`) to also cover `.txt` — i.e. add `!docs/qa/**/*.*txt`, mirroring the `.log` rescue exactly. This is the narrowest plausible general fix: unlike the blanket `!docs/qa/**` the prior session already rejected, an extension-scoped rescue cannot interact with any of the project’s genuinely extension-based secret patterns (`*.key`, `*.pem`, `*.p12`, `*.pfx`, `*.jks`, `*.env` — none of these end in `.txt`, so a `.txt`-scoped negation can never rescue them).

Measured, and found unsafe. This project’s OWN `.gitignore` documents a secret-naming convention that specifically uses the `.txt` extension: `cookies_*.txt` (line 360, “§11.4.10 CRITICAL — cookies txt files MUST NEVER be git-versioned”). The existing `test_gitignore_swallow_is_loud.sh` golden-false corpus independently confirms `.txt` is a live secret-bearing shape in this project’s own threat model — it includes `cookies_rutacker.txt`, `RUTACKER_session.txt`, and `my_password.txt` as entries that MUST stay ignored. A scratch-tree experiment, run exactly in the style of the prior session’s own `.env/creds.json` disproof of the blanket-directory candidate, confirms the `.txt`-extension candidate would leak exactly these shapes if placed under `docs/qa/**`:

```
$ # scratch git tree, real .gitignore + hypothetical "!docs/qa/**/*.*txt"
$ printf 'cookie=SECRET_SESSION_TOKEN\n' > docs/qa/FAKE/evidence/cookies_rutacker.txt
$ printf 'user=admin\npass=hunter2\n' > docs/qa/FAKE/evidence/my_password.txt
$ printf 'harmless observation\n' > docs/qa/FAKE/evidence/loginctl_user_state.txt
$ git status --porcelain --untracked-files=all
?? .gitignore
?? docs/qa/FAKE/evidence/cookies_rutacker.txt
?? docs/qa/FAKE/evidence/loginctl_user_state.txt
?? docs/qa/FAKE/evidence/my_password.txt
```

All three become visible/stageable under the hypothetical rule — including the two genuinely secret-shaped files. This is a real, non-hypothetical regression risk for THIS project specifically (not a generic “any glob can theoretically leak” caveat): `.txt` is this repo’s documented cookie-file extension, so a `.txt`-wide rescue under `docs/qa/**` is measurably LESS safe than the already-shipped `.log`-wide rescue (`.log` matches none of this project’s documented secret-file conventions).

Decision: do not ship this fix. Per the task’s explicit instruction — “Do not weaken or blanket-disable the `*credentials*` or `*_user*` gitignore patterns themselves... never a broad exception that could let a real credential leak through” — and per this session’s authorized scope (only `.gitignore` plus the three named artifacts; no second new script was authorized for a QA-evidence-specific detective gate, and the prior session’s own considered-but-rejected “narrower re-deny of the word-substring globs scoped to `docs/qa/**`” direction remains rejected for the same reason it was originally: this project routinely authors legitimate auth-testing QA evidence whose SUBJECT is credentials).

.gitignore was left untouched by this session (confirmed: `git diff --stat -- .gitignore` and `git diff --stat HEAD -- .gitignore` both produced no output). Criterion (3)’s “future-swallow” mechanism remains closed ONLY for the `.md-export-twin` class (§3a); it remains open for the QA-evidence non-`.log`-extension class, now with one more concretely-disproven candidate direction on record.

4. Verification summary

Check	Command	Result
<code>.gitignore</code> BEFORE/ AFTER (unchanged this session)	<code>git check-ignore -v <3 paths></code>	Only <code>loginctl_user_state.tx</code> still matches a rule (<code>.gitignore:73:*_user*</code>) the <code>html/pdf</code> twins match none
<code>html/pdf</code> twins tracked	<code>git ls-files docs/guides/tracker-credentials.{html,pdf}</code>	Both listed — tracked, clean
<code>html/pdf</code> twins stageable	<code>git add -n docs/guides/tracker-credentials.{html,pdf}</code>	No output (already committed and unmodified — the strongest form of “committable”)
<code>.gitignore</code> untouched	<code>git diff --stat -- .gitignore / git diff --stat HEAD -- .gitignore</code>	Empty (no diff)
<code>BOB-124.txt</code> file	<code>ls, git log --all, find .</code>	Confirmed absent on disk and in all of history — nothing to commit
New guard, standalone		OK ... (436 <code>.md</code> files scanned), exit 0

Check	Command	Result
	bash scripts/pre_build/ check_md_export_twins_committable.sh "\$ (pwd) "	
New guard, syntax	bash -n scripts/pre_build/ check_md_export_twins_committable.sh	SYNTAX_OK
New test, syntax	bash -n tests/pre_build/ test_check_md_export_twins_committable.sh	SYNTAX_OK
		RESULT: PASS (exit 0)
New test, full run	bash tests/pre_build/ test_check_md_export_twins_committable.sh	— all 7 arms, RED+GREEN+golden- false×2+fail-closed+real- repo+\$1.1 mutation
.txt-extension rescue candidate	scratch git-tree experiment	DISPROVEN — leaks cookies_*.txt-shaped secrets; not shipped

5. Assumptions made (item text did not fully specify)

1. **“§11.4.65 scope” for the new guard** is operationally defined as exactly the scope `scripts/generate_markdown_exports.sh` itself sweeps (`root/*.md + docs/**/*.md + scripts/**/*.md`), since that script IS the mechanism §11.4.65 mandates, rather than re-deriving scope from the constitution’s prose independently. This keeps the new guard and the generator in permanent agreement about what “in scope” means.
2. **The guard checks .html, .pdf, AND .docx** twins, even though the item text names only `.html/.pdf` for the specific `tracker-credentials` instance. §11.4.65/§11.4.153 mandate all three as the export set, and BOB-225 (already closed) establishes `.docx` as a live member of this same defect class, so covering all three closes more of “this class” per criterion (3)’s intent without expanding the file-scope boundary (still one new script + one new test).
3. **A twin that does not exist on disk is never flagged** by the new guard — the item’s phrasing (“has COMMITTABLE twins”) is read as conditional on existence, not as also mandating presence (a separate, already-owned concern per BOB-223/CM-MARKDOWN-EXPORT-SYNC). Conflating the two would make this guard fire on the ~large fraction of in-scope `.md` files whose exports simply have not been regenerated recently, which is not this item’s defect class.

4. **No companion doc was created under docs/scripts/** for the new script, even though the sibling `check_gitignore_swallow.sh` (BOB-212) has one. This is deliberate, not an oversight: `docs/scripts/` was not in this session's authorized file scope, AND creating one would directly reproduce the exact hazard `docs/Issues.md` BOB-223 already documents (a new `docs/scripts/<name>.md` companion doc trips CM-MARKDOWN-EXPORT-SYNC for its own un-generated `.html/.pdf` twins). The script instead carries a thorough in-source header (Purpose / Usage / Behaviour / Exit codes / Depends / Refs), matching §11.4.18's in-source-block half.
5. **Criterion (2) is read as genuinely unsatisfiable**, not as "left for a future session to eventually satisfy" — there is no plausible path to making a nonexistent, never-committed file trackable other than fabricating its content, which this session declines to do.

6. What is NOT done (for the conductor / next session)

- Criterion (2) — permanently unsatisfiable for this specific artifact; no further action possible without fabricating evidence.
- Criterion (3)'s QA-evidence half — the `.log`-extension rescue remains the only safe general mechanism shipped; extending it to cover more non-secret-shaped QA-evidence extensions (`.txt` specifically disproven this session) needs its own narrower design — candidate directions not yet disproven: (a) a detective (loud-refusal, non-rescuing) gate scoped to `docs/qa/**`, mirroring `check_gitignore_swallow.sh`'s "leave the deny rules untouched, only add visibility" approach instead of a rescue-based one; (b) a content-inspection check rather than a path-pattern one. Both were out of this session's authorized scope (no second new script was authorized).
- **Wiring `check_md_export_twins_committable.sh` into `scripts/pre_build_verification.sh` is NOT done** — per the explicit scope boundary, that file was never touched. The invariant call needed is: a new numbered invariant that runs `scripts/pre_build/check_md_export_twins_committable.sh "$PROJECT_ROOT"` (or equivalent repo-root variable already used by sibling invariants in that file, e.g. however invariant 56/CM-GITIGNORE-SWALLOW-GUARD for `check_gitignore_swallow.sh` is currently wired) and BLOCKS the build on a non-zero exit, printing the guard's own stdout/stderr (which already names the exact swallowed file + source `.md` + blocking `.gitignore` line — no additional formatting needed at the call site).

- The DB item itself was left untouched (no `workable-items` CLI invocation, no direct edit to `docs/workable_items.db`), per this session's instructions — closure/status change is for the conductor.